

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Ingegneria Gestione ed Evoluzione del Software

CodeFace4Smell

Repository: <https://github.com/rosacarota/CodeFace4Smell>

Project Report

Prof.
Andrea De Lucia

Dott.ssa
Giusy Annunziata

Studenti:
Rosa Carotenuto
MAT.: 0522502024
Luigi Guida
MAT.: 0522502026

ANNO ACCADEMICO 2025/2026

CONTENTS

Contents

1	Introduzione	1
1.1	Contesto del progetto	1
1.2	CodeFace4Smell e i community smells	1
1.3	Architettura del sistema	1
1.4	Stato iniziale e motivazioni del progetto	1
2	Evoluzione degli script di avvio e provisioning	3
2.1	Contesto e obiettivi	3
2.2	Approccio metodologico: dalla “compatibilità storica” alla “ripetibilità moderna”	3
2.3	Aggiornamento dell’infrastruttura Vagrant	3
	Migrazione della base VM.	3
	Robustezza del boot e compatibilità del provider.	3
	Rete e porte.	4
2.4	Riorganizzazione della pipeline: maggiore controllo e separazione dei passi	4
	Invocazione esplicita degli script.	4
	Provisioning e test come fasi distinte.	4
2.5	Ottimizzazione del setup R: caching persistente e installazione controllata	4
	Criticità iniziale.	4
	Hardening della cache e coerenza di configurazione.	4
2.6	Aggiornamento di Node, dashboard e migrazione Python 2 → Python 3	4
	Node: aggiornamento della versione e installazione deterministica.	4
	Migrazione Python 2 → Python 3: intervento sul codice e sulle dipendenze.	5
2.7	Database: da stack legacy a configurazione moderna e stabile	5
	Configurazione attuale.	5
	Provisioning: rafforzamento della procedura di setup.	6
	Porting di cppstats a Python 3 tramite 2to3 e revisione manuale.	6
2.8	Risultati e discussione delle scelte	6
3	Ripristino della test suite e adeguamento del codice	7
3.1	Contesto e obiettivo	7
3.2	Stato iniziale: perché i test non erano eseguibili	7
3.3	Interventi sul codice Python a supporto dei test	7
3.4	Standardizzazione dell’esecuzione con pytest e adeguamento della suite	7
	Migrazione e compatibilità Python 2 → Python 3.	7
3.5	Interventi sui componenti R richiamati dai test	7
3.6	Gestione dei test già disabilitati nel repository	7
3.7	Risultato	8
4	Avvio e configurazione del dashboard Shiny	9
4.1	Scopo del dashboard e architettura attesa	9
4.2	Struttura della directory Shiny e configurazione del database	9
4.3	Modifiche per l’avvio corretto della dashboard	9
4.4	Verifica del funzionamento della dashboard Shiny	9

5	Containerizzazione con Docker	11
5.1	Contesto e motivazioni	11
5.2	Struttura generale dell'ambiente containerizzato	11
5.3	Database: inizializzazione controllata e persistenza	11
5.4	Immagine applicativa: consolidamento dell'ambiente di esecuzione	11
5.5	Gestione delle librerie R: riuso e riduzione dei tempi di setup	12
5.6	Risultato	12
6	Conclusioni e sviluppi futuri	13
6.1	Esempi di output e interfaccia grafica	13
6.2	Integrazione continua e possibili estensioni	14

1 Introduzione

1.1 Contesto del progetto

Codeface è un framework open source progettato per l'analisi socio-tecnica di progetti software. Il tool integra informazioni provenienti da diversi canali (repository Git, mailing list, issue tracker) per costruire *Developer Social Networks* (DSN) e metriche sull'evoluzione del progetto, sulle collaborazioni tra sviluppatori e sulla struttura organizzativa della community. Su questa base, *CodeFace4Smell* estende le funzionalità di *Codeface* con l'obiettivo specifico di rilevare automaticamente *community smells*, ovvero pattern ricorrenti nella struttura sociale e organizzativa di una comunità di sviluppo che possono portare, nel lungo periodo, a problemi di coordinamento, aumento del *social debt* e, indirettamente, a debito tecnico nel codice.

1.2 CodeFace4Smell e i community smells

Nel lavoro di Tamburri, Palomba e Kazman viene presentato *CodeFace4Smells* come estensione di *Codeface* in grado di identificare quattro tipi di community smells all'interno dei Developer Social Networks estratti dai progetti: Organisational Silo Effect, Lone Wolf Effect, Black Cloud Effect e Bottleneck (Radio-silence) Effect.

Dal punto di vista implementativo, *CodeFace4Smell* sfrutta i grafi di comunicazione e di collaborazione costruiti da *Codeface* per individuare tali smells tramite regole basate su teoria dei grafi e social network analysis. A partire dai log dei repository Git e dai canali di comunicazione, il sistema costruisce reti di relazioni tra sviluppatori e identifica pattern micro-strutturali che corrispondono alle definizioni formali dei community smells.

1.3 Architettura del sistema

Dal punto di vista architetturale, il progetto *CodeFace4Smell* è composto da più componenti integrati, ciascuno con un ruolo specifico nella pipeline di analisi:

- **Backend Python:** responsabile dell'analisi dei repository Git, dell'estrazione delle metriche tecniche e sociali e della costruzione dei Developer Social Networks. In questa fase vengono anche invocati strumenti di analisi statica esterni (ad es. *cppstats*) per arricchire il set di feature utilizzate nelle analisi successive.
- **Componenti R:** implementano la logica di analisi statistica, i metodi di clustering e le regole di rilevazione dei community smells, producendo tabelle e serie temporali che descrivono l'evoluzione della community e dei suoi pattern organizzativi.
- **Database relazionale:** fornisce lo storage persistente per i dati estratti e le analisi derivate.
- **Dashboard Shiny (R):** interfaccia web interattiva che costituisce il principale punto di accesso ai risultati di analisi, permettendo a ricercatori e sviluppatori di esplorare metriche di collaborazione, comunicazione e qualità organizzativa della community tramite widget, grafici e viste aggregate.

1.4 Stato iniziale e motivazioni del progetto

La codebase di *CodeFace4Smell* deriva da uno stack tecnologico storicizzato (intorno al 2013–2014), con un'infrastruttura di deployment basata su Vagrant, una macchina virtuale

Ubuntu 12.04 e dipendenze ormai deprecate (Python 2, versioni legacy di R e Node.js, repository di pacchetti non più mantenuti). Nel contesto attuale, tali scelte si traducono in una bassa affidabilità del provisioning, nella difficoltà di ricreare l'ambiente senza interventi manuali e nell'impossibilità di eseguire la test suite e la dashboard Shiny su sistemi moderni.

In particolare, lo stato iniziale del progetto era caratterizzato da:

- **Infrastruttura obsoleta:** dipendenza da una VM legacy, script di provisioning fragili e forte dipendenza da risorse esterne non più disponibili.
- **Test suite non funzionante:** presenza di test unitari e di integrazione, ma non eseguibili in un ambiente moderno a causa di incompatibilità di runtime.
- **Dashboard Shiny non operativa:** problemi di connessione al database e configurazioni non portabili che impedivano il corretto avvio e il rendering delle viste principali della dashboard.

Alla luce di queste criticità, il progetto svolto nel corso si è posto come obiettivo principale la **modernizzazione e stabilizzazione** di CodeFace4Smell, senza stravolgerne il modello concettuale, ma rendendolo nuovamente eseguibile in un contesto tecnologico attuale. In particolare, il lavoro si è concentrato su:

- Migrazione dell'infrastruttura da Vagrant basato su Ubuntu 12.04 a un ambiente moderno, con provisioning robusto e ripetibile.
- Aggiornamento dello stack tecnologico (Python 2 → Python 3, ecosistema R, Node.js, database MySQL/MariaDB) e adeguamento del codice e delle dipendenze.
- Ripristino ed esecuzione stabile della test suite esistente, in modo da utilizzare i test come strumento di validazione delle modifiche introdotte.
- Configurazione e avvio affidabile della dashboard Shiny, collegato ai database di analisi, per consentire un'esplorazione interattiva dei risultati.

Il presente report documenta il lavoro di recupero e modernizzazione svolto sul progetto, descrivendo per ciascun ambito le problematiche riscontrate, le soluzioni adottate e i risultati ottenuti.

2 Evoluzione degli script di avvio e provisioning

2.1 Contesto e obiettivi

La VM originale era basata su Ubuntu 12.04 (precise64) e lo stack dipendeva da componenti non più mantenuti (Python 2, repository PPA datati, versioni legacy di Node e tool esterni distribuiti tramite tarball o binari non più reperibili). In fase iniziale, tali vincoli si traducevano in una scarsa affidabilità del provisioning: `vagrant up` poteva andare in timeout, le installazioni fallivano per dipendenze non più disponibili e la riproducibilità era limitata.

Un problema particolarmente rilevante era rappresentato dall’ecosistema R. Nella configurazione originaria, molte librerie venivano reinstallate e spesso compilate da sorgente ad ogni creazione della VM, con tempi elevati e una maggiore probabilità di errore (dipendenze native, compilazioni lunghe, instabilità in caso di interruzioni).

Alla luce di queste criticità, il lavoro sugli script di avvio e provisioning ha perseguito due obiettivi principali:

- (1) **Riproducibilità e stabilità**: ottenere un ambiente che possa essere creato, distrutto e ricreato con `vagrant up` senza interventi manuali, minimizzando le differenze tra provisioning successivi.
- (2) **Riduzione dei tempi di rebuild**: evitare reinstallazioni e ricompilazioni inutili, mantenendo invariato il set di dipendenze richiesto dal progetto, con particolare attenzione all’installazione di R.

2.2 Approccio metodologico: dalla “compatibilità storica” alla “ripetibilità moderna”

La strategia adottata non è stata una riscrittura completa del progetto, ma un intervento mirato sugli elementi che impedivano l’esecuzione in un contesto moderno. In particolare, si è scelto di:

- mantenere la struttura complessiva del provisioning come pipeline a step (repository → dipendenze comuni → R → Node → Python → tool esterni → database);
- aggiornare i singoli step per eliminare assunzioni implicite (servizi già avviati, pacchetti sempre disponibili, percorsi dipendenti dalla working directory);
- introdurre meccanismi di robustezza (controlli, retry, idempotenza) per ridurre failure intermittenti.

2.3 Aggiornamento dell’infrastruttura Vagrant

Migrazione della base VM. Il primo intervento necessario è stata la migrazione della base da precise64 (Ubuntu 12.04) a bento/ubuntu-20.04. Tale passaggio è stato indispensabile per rientrare in un ecosistema supportato: repository APT attivi e disponibilità di pacchetti in versioni compatibili. In assenza di questa migrazione, il provisioning risultava strutturalmente fragile, poiché dipendente da risorse esterne non più mantenute.

Robustezza del boot e compatibilità del provider. Sono state introdotte impostazioni per ridurre failure durante l’avvio della VM (ad esempio gestione del `boot_timeout` e della connessione SSH). Inoltre sono state aumentate le risorse assegnate (RAM e CPU), motivando

questa scelta con la necessità di sostenere installazioni e compilazioni più pesanti rispetto al contesto originario. In parallelo, alcune opzioni del provider VirtualBox sono state rese esplicite per migliorare stabilità e ridurre comportamenti intermittenti, soprattutto in presenza di cartelle condivise e I/O intensivo.

Rete e porte. Sono state mantenute le porte operative principali del progetto (dashboard e test), e si è adottata una configurazione più resiliente del forwarding (in particolare per SSH, con meccanismi di auto-correzione).

2.4 Riorganizzazione della pipeline: maggiore controllo e separazione dei passi

Invocazione esplicita degli script. Nella versione originale, l'invocazione degli script assumeva permessi e contesto coerenti. Per ridurre ambiguità, l'esecuzione è stata resa esplicita tramite bash. Questo intervento aumenta la determinazione del provisioning e riduce errori legati a differenze tra ambienti.

Provisioning e test come fasi distinte. Un'ulteriore scelta progettuale è stata separare la creazione dell'ambiente dall'esecuzione dei test. Nel setup originario, un test fallito poteva interrompere la creazione della VM, anche quando l'ambiente era sostanzialmente corretto ma non ancora completamente stabilizzato. Nel workflow aggiornato, i test rimangono disponibili ma vengono eseguiti in un secondo momento, consentendo di ottenere prima un ambiente "avviabile" e poi una validazione controllata.

2.5 Ottimizzazione del setup R: caching persistente e installazione controllata

Criticità iniziale. Durante i primi provisioning, l'installazione dei pacchetti R è emersa come il principale collo di bottiglia. Molte dipendenze venivano reinstallate e compilate da sorgente, portando a tempi molto elevati e aumentando la probabilità di errori. Questa criticità era particolarmente rilevante perché la VM doveva essere ricreata frequentemente compromettendo l'intero ciclo di sviluppo e debugging.

Il vincolo del progetto era mantenere il set di dipendenze; pertanto, la soluzione ricercata non era ridurre i pacchetti, ma evitare la ripetizione di operazioni costose. Da qui la scelta di introdurre una **cache persistente delle librerie R** tramite cartella condivisa tra host e guest:

```
./rlibs (host) → /opt/Rlibs (guest)
```

La cache permette di riutilizzare pacchetti già compilati tra ricreazioni successive della VM, riducendo drasticamente i tempi di rebuild.

Hardening della cache e coerenza di configurazione. Per rendere la cache effettiva non è sufficiente conservarla: è necessario che R la utilizzi come percorso prioritario. Per questo sono stati introdotti accorgimenti di configurazione e aggiornamenti agli script di installazione in modo da evitare reinstallazioni inutili quando i pacchetti risultano già presenti.

2.6 Aggiornamento di Node, dashboard e migrazione Python 2 → Python 3

Node: aggiornamento della versione e installazione deterministica. Nel progetto originale l'ecosistema Node era vincolato a versioni storiche non più supportate e, soprattutto,

l'installazione assumeva implicitamente che l'ambiente Node/npm fosse già presente nella VM.

Per rendere il setup riproducibile, si è aggiornato il provisioning adottando una versione **LTS moderna di Node.js** e rendendo lo script `install_codeface_node.sh` autosufficiente:

- verifica esplicita della presenza di node e npm;
- installazione automatica di Node.js in caso di assenza;
- esecuzione di `npm install` nel path corretto del servizio (`id_service`), evitando dipendenze dalla working directory.

Questa modifica riduce gli errori “dipende dall'ambiente” e rende più stabile l'avvio dei componenti Node durante provisioning e testing.

Migrazione Python 2 → Python 3: intervento sul codice e sulle dipendenze. Una parte sostanziale del lavoro è stata la migrazione dell'intero runtime applicativo da **Python 2** a **Python 3**, necessaria perché Python 2 e molte librerie correlate risultano deprecate e non più disponibili/compatibili in ambienti moderni. La migrazione ha richiesto sia aggiornamenti del processo di installazione, sia modifiche puntuali al codice e alle dipendenze.

Dal punto di vista del provisioning, l'installazione è stata aggiornata a toolchain Python 3:

- installazione di python3 e pacchetti python3-* ;
- adozione di pytest per l'esecuzione dei test.

Dal punto di vista delle dipendenze applicative, sono stati sostituiti pacchetti non compatibili con Python 3 con alternative mantenute. In particolare:

- `progressbar` → `progressbar2`;
- `python-ctags` → `python-ctags3`;
- driver DB legacy (`MySQL-python/MySQL_python`) → `pymysql`.

2.7 Database: da stack legacy a configurazione moderna e stabile

Nel progetto originale, la componente database era stata progettata per l'ambiente di riferimento dell'epoca (Ubuntu 12.04). In quel contesto, lo stack MySQL disponibile tramite i repository di sistema corrispondeva tipicamente alla generazione MySQL 5.5; di conseguenza, sia lo schema sia gli script di inizializzazione erano calibrati su presupposti coerenti con quel contesto.

Con la migrazione del provisioning su Ubuntu 20.04, tale impostazione non ha più garantito lo stesso livello di affidabilità. L'obiettivo era di assicurare che il database risultasse **installabile, inizializzabile e riutilizzabile in modo ripetibile** all'interno di un sistema moderno. In questo scenario, differenze tra versioni e impostazioni di default (nonché tempi e modalità di avvio del servizio) possono generare anomalie intermittenti: ad esempio import eseguiti prima che il DB sia pronto, riavvii non completamente puliti o residui di stato tra un provisioning e il successivo.

Configurazione attuale. L'assetto attuale adotta un database pienamente compatibile con Ubuntu 20.04, MariaDB, e una procedura di inizializzazione più controllata. Oltre alla configurazione del servizio, è stato necessario intervenire sullo schema per allinearli

al nuovo contesto. In particolare, lo schema è stato aggiornato adottando un encoding moderno.

Provisioning: rafforzamento della procedura di setup. Lo script di setup del database è stato quindi revisionato con un approccio più robusto: avvio controllato del servizio, attesa fino alla disponibilità effettiva e gestione esplicita di condizioni anomale (ad esempio servizio non responsivo o stato non pulito). Inoltre, la creazione del database applicativo e del database di test è stata resa ripetibile e l'import dello schema è stato impostato in modo da poter essere rilanciato senza interventi manuali.

Porting di cppstats a Python 3 tramite 2to3 e revisione manuale. All'interno di *Code-Face4Smell*, cppstats è utilizzato come tool di analisi statica. Il suo compito principale è estrarre in modo automatico un insieme di indicatori strutturali, che vengono poi impiegati come input per le fasi successive della pipeline di analisi. In questo senso, cppstats rappresenta un componente di supporto che contribuisce a costruire le feature e le metriche necessarie.

Dal punto di vista operativo, l'uso di cppstats avviene come step integrato nel workflow di provisioning/esecuzione: una volta reso disponibile nell'ambiente (installazione e configurazione delle dipendenze richieste), il tool viene invocato sui repository di progetto da analizzare, producendo output strutturati utilizzabili dagli altri componenti del sistema.

Per rendere cppstats compatibile con l'ambiente moderno adottato nel progetto, è stato necessario migrare il tool da Python 2 a Python 3. La migrazione è stata avviata utilizzando 2to3, che ha permesso di automatizzare la conversione delle principali incompatibilità sintattiche (ad esempio l'aggiornamento di costrutti e funzioni non più supportati in Python 3).

La conversione automatica ha rappresentato un primo passo, ma non è stata sufficiente da sola: sono stati quindi effettuati interventi manuali su alcuni script per ripristinare il corretto funzionamento nel nuovo runtime, gestire differenze comportamentali (ad esempio nella gestione di stringhe e output) e riallineare l'esecuzione alle dipendenze e ai percorsi previsti dal provisioning.

L'insieme delle modifiche è stato consolidato in un fork del repository, così da mantenere una sorgente controllata.

2.8 Risultati e discussione delle scelte

Dal punto di vista dei trade-off, l'approccio scelto ha comportato una maggiore articolazione degli script. Tuttavia, tale complessità è giustificata dal requisito principale del progetto: garantire che l'ambiente sia ricreabile, anche in presenza di variabilità tipiche dei sistemi legacy e delle loro dipendenze.

3 Ripristino della test suite e adeguamento del codice

3.1 Contesto e obiettivo

Il progetto includeva già una test suite (unit e integration test), ma nello stato iniziale non risultava eseguibile nell'ambiente modernizzato. L'obiettivo di questa fase non è stato introdurre nuovi test, bensì ripristinare la possibilità di eseguire quelli esistenti e utilizzare la suite come strumento di validazione delle modifiche introdotte durante il reengineering dell'ambiente.

3.2 Stato iniziale: perché i test non erano eseguibili

La causa principale che impediva l'esecuzione della test suite era riconducibili all' **Incompatibilità di runtime**: parte del codice applicativo e di alcuni tool integrati era ancora basata su Python 2, quindi non compatibile con l'ecosistema moderno adottato e con le relative dipendenze aggiornate.

3.3 Interventi sul codice Python a supporto dei test

Per rendere la suite eseguibile in modo stabile, sono stati introdotti interventi di adeguamento al runtime Python 3 e di consolidamento di alcune parti dell'applicazione che venivano attraversate dai test. Le modifiche principali hanno riguardato:

3.4 Standardizzazione dell'esecuzione con pytest e adeguamento della suite

Per rendere la validazione più replicabile e coerente con le pratiche attuali dell'ecosistema Python, l'esecuzione dei test è stata standardizzata tramite pytest.

L'adozione di quest'ultimo ha richiesto un adeguamento mirato di parte della test suite, in particolare dei test di integrazione, affinché risultassero compatibili.

Migrazione e compatibilità Python 2 → Python 3. Sono stati aggiornati costrutti e porzioni di codice non compatibili con Python 3 (ad esempio sintassi di gestione eccezioni e funzioni non più supportate), adottando un approccio conservativo volto a preservare la semantica originale. In parallelo, sono state aggiornate dipendenze non più mantenute o non compatibili con Python 3, sostituendole con alternative moderne equivalenti.

3.5 Interventi sui componenti R richiamati dai test

Ove i test coinvolgono componenti in R, gli interventi si sono concentrati sull'assicurare la ripetibilità dell'esecuzione (dipendenze risolte, percorsi corretti, librerie disponibili e coerenti). Questa parte si collega direttamente alle scelte di provisioning già discusse (cache delle librerie).

3.6 Gestione dei test già disabilitati nel repository

È importante notare che alcuni test risultavano già commentati o disabilitati nella repository originale. Tale scelta è stata mantenuta per preservare la coerenza e per evitare instabilità della suite. Nel contesto del progetto, la priorità è stata rendere affidabile l'esecuzione del sottoinsieme di test effettivamente utilizzabile.

3.7 Risultato

Il risultato principale di questa fase è stato il ripristino di una test suite eseguibile in un ambiente moderno, tale da fornire un riscontro concreto sull'effettiva operatività del sistema dopo le modifiche di provisioning e porting. In questo modo i test sono tornati a rappresentare uno strumento di validazione utile e ripetibile, anziché un blocco iniziale che impediva di verificare il comportamento del progetto.

4 Avvio e configurazione del dashboard Shiny

4.1 Scopo del dashboard e architettura attesa

La dashboard Shiny di CodeFace4Smell costituisce il principale punto di accesso per la visualizzazione dei risultati di analisi sui repository Git, offrendo una panoramica interattiva sull'evoluzione del progetto e sulla qualità della comunità di sviluppo.

Dal punto di vista architetturale, la dashboard è realizzato come applicazione web Shiny che combina un layout a widget (drag-and-drop). L'applicazione principale è distribuita in `codeface/R/shiny/apps/dashboard/` ed è organizzata in pannelli e widget coordinati che aggregano le informazioni provenienti dalle analisi eseguite dai tool di CodeFace4Smell.

4.2 Struttura della directory Shiny e configurazione del database

La componente Shiny è organizzata in un albero di directory dedicato sotto `codeface/R/shiny`. La directory radice contiene una pagina statica `index.html`, responsabile dell'instradamento delle richieste HTTP in ingresso verso le diverse applicazioni disponibili, e una sottodirectory `apps/` che ospita le varie istanze Shiny. La dashboard principale è implementato in `apps/dashboard/` attraverso la classica separazione in componenti di interfaccia, logica di server e inizializzazione globale, affiancati da file di configurazione per i widget e sotto-applicazioni dedicate ad aspetti specifici (dettagli, grafici, progetti).

Dal lato backend, la dashboard comunica con un'istanza MariaDB che espone due database logici: `codeface` per le analisi di produzione e `codeface_testing` per le esecuzioni di test. La procedura di provisioning crea gli utenti necessari e inizializza lo schema a partire da `codeface_schema.sql`, così che la dashboard possa interrogare metadati di progetto, risultati di clustering, serie temporali e altre statistiche immediatamente dopo l'avvio della macchina virtuale.

4.3 Modifiche per l'avvio corretto della dashboard

Per garantire che il Shiny Server avviasse direttamente la dashboard principale di CodeFace4Smell, sono state applicate alcune modifiche mirate alla configurazione e ai file del progetto. In primo luogo, è stata razionalizzata la struttura della directory Shiny, disattivando i vecchi file di test e mantenendo come entry point logico il percorso `apps/dashboard`, in modo che le richieste in arrivo vengano instradate verso la dashboard di analisi invece che verso applicazioni ausiliarie interne.

4.4 Verifica del funzionamento della dashboard Shiny

Dopo l'applicazione delle modifiche di configurazione, è stata condotta una procedura di test per validare l'intero ciclo di avvio della dashboard Shiny, dall'ingestione dei dati fino al rendering dell'interfaccia. In questa fase sono stati risolti anche alcuni aspetti operativi (come la normalizzazione dei line ending negli script di provisioning) per garantire l'esecuzione corretta degli script in ambiente Linux.

Per la verifica, è stata eseguita un'analisi su un repository reale (*toybox*), utilizzando la configurazione globale di CodeFace4Smell e una specifica configurazione di progetto dedicata. Al termine dell'analisi, il progetto è risultato correttamente registrato nel database `codeface`, e la dashboard è stata avviata sulla macchina virtuale ed esposto all'host tramite

la porta 8081. Durante la validazione manuale sono stati verificati il popolamento del menu “Codeface projects”, la selezione del progetto analizzato e il caricamento dei principali widget, inclusa la connessione attiva al database e l’aggiornamento delle viste di sintesi sull’evoluzione del repository.

5 Containerizzazione con Docker

5.1 Contesto e motivazioni

In parallelo alla stabilizzazione dell'ambiente basato su Vagrant, è stata introdotta una versione containerizzata del progetto tramite Docker. La motivazione principale è stata affiancare alla VM una soluzione più **portabile, riproducibile e facilmente ricostruibile**, in cui ambiente di esecuzione e servizi risultino descritti in modo dichiarativo e avviabili in modo coerente su host differenti, senza dipendere da configurazioni locali o installazioni manuali.

Nel caso di un progetto legacy come *CodeFace4Smell*, la containerizzazione si è rivelata particolarmente utile per tre motivi principali:

- **Riproducibilità dell'ambiente:** l'intero sistema è definito rendendo la ricostruzione dell'ambiente deterministica.
- **Isolamento delle dipendenze:** runtime e librerie (Python, R, Node e tool esterni) sono incapsulati nei container, limitando conflitti con l'host e comportamenti dipendenti dalla macchina.
- **Chiarezza architetturale:** database, applicazione e dashboard vengono separati in servizi distinti.

5.2 Struttura generale dell'ambiente containerizzato

L'orchestrazione dell'ambiente Docker è gestita tramite `docker-compose` e prevede tre servizi principali, avviabili in modo coordinato:

- (1) un servizio **database** (`codeface-db`);
- (2) un **container applicativo** principale (`codeface-app`);
- (3) un servizio separato dedicato alla **dashboard** (`codeface-shiny`).

I servizi condividono una rete dedicata e sono configurati con politiche di restart, migliorando la stabilità complessiva del sistema.

5.3 Database: inizializzazione controllata e persistenza

Il database è fornito tramite un container basato su MariaDB 10.11. L'inizializzazione avviene in modo automatico e dichiarativo, tramite variabili d'ambiente e script montati, che si occupano della creazione del database e dell'utente applicativo.

Un aspetto rilevante è la **persistenza dei dati**: lo stato del database viene mantenuto tramite un volume Docker dedicato, evitando la perdita delle informazioni a ogni riavvio dei container. Per garantire un avvio ordinato del sistema, è stato inoltre introdotto un meccanismo di **healthcheck** che assicura che i servizi applicativi vengano avviati solo quando il database è effettivamente pronto.

5.4 Immagine applicativa: consolidamento dell'ambiente di esecuzione

Il cuore dell'ambiente containerizzato è rappresentato dall'immagine applicativa, costruita tramite un Dockerfile multi-stage. Questa scelta consente di separare una fase di **build** — dedicata alla compilazione di dipendenze native — da una fase di **runtime**.

All'interno dell'immagine finale vengono installati:

- runtime **Python 3**, configurato per l'esecuzione del progetto e dei test;
- ambiente **R**, con supporto a pacchetti complessi e grafici;
- **Node.js 18 LTS**, necessario per componenti web e strumenti di supporto;
- tool esterni richiesti dalle analisi, come Graphviz, Doxygen e ctags;
- supporto a **Shiny Server** per l'esecuzione della dashboard.

Questa struttura riproduce in modo più affidabile un ambiente che, nel provisioning tradizionale su VM, risultava fragile e particolarmente oneroso in termini di tempo.

5.5 Gestione delle librerie R: riuso e riduzione dei tempi di setup

Coerentemente con l'esperienza maturata nella VM, particolare attenzione è stata posta alla gestione delle librerie R, che rappresentano uno dei principali fattori di costo nel provisioning. L'immagine distingue tra una libreria di base "baked" nell'immagine e una libreria persistente montata tramite volume, permettendo di riutilizzare i pacchetti già installati tra esecuzioni successive.

5.6 Risultato

La containerizzazione ha introdotto un secondo canale di esecuzione complementare a Vagrant, orientato a portabilità, riproducibilità e chiarezza architetturale. L'uso di Docker e Docker Compose ha permesso di descrivere in modo esplicito sia l'ambiente applicativo sia l'orchestrazione dei servizi, riducendo dipendenze implicite dall'host e fornendo una base solida per future integrazioni di testing automatizzato e pratiche di CI/CD.

6 Conclusioni e sviluppi futuri

Il lavoro svolto ha avuto come obiettivo principale il ripristino dell'eseguitività di *CodeFace4Smell* in un contesto tecnologico moderno, intervenendo in modo mirato sull'infrastruttura, sul provisioning e sulla compatibilità dei componenti software, senza alterare il modello concettuale e le finalità originali dello strumento.

Attraverso la modernizzazione dell'ambiente basato su Vagrant, la migrazione dei runtime obsoleti (in particolare Python 2 → Python 3), il rafforzamento degli script di provisioning e il ripristino della test suite, è stato possibile rendere nuovamente il progetto avviabile, testabile e utilizzabile su sistemi attuali. Un risultato rilevante è stato anche il recupero del dashboard Shiny, che rappresenta il principale punto di accesso ai risultati di analisi socio-tecnica e costituisce un elemento chiave per la fruizione dei dati prodotti da *CodeFace4Smell*.

In parallelo, l'introduzione di una versione containerizzata tramite Docker ha affiancato l'approccio basato su macchina virtuale con una soluzione più portatile e riproducibile. La definizione dichiarativa dei servizi e dell'ambiente di esecuzione ha consentito di ridurre la dipendenza da configurazioni implicite e di fornire una base più solida per l'evoluzione futura del progetto.

6.1 Esempi di output e interfaccia grafica

Al termine delle attività di modernizzazione è stato possibile, come già detto, avviare correttamente anche l'interfaccia grafica del sistema basata su Shiny.

In questa sezione vengono riportate alcune schermate esemplificative dell'interfaccia, con il solo scopo di mostrare:

- il corretto avvio dell'applicazione;
- la connessione ai database di analisi;
- la disponibilità dei risultati prodotti dal backend.

Le immagini rappresentano una dimostrazione visiva del fatto che il sistema, nel suo complesso, risulta correttamente eseguibile e integrato.

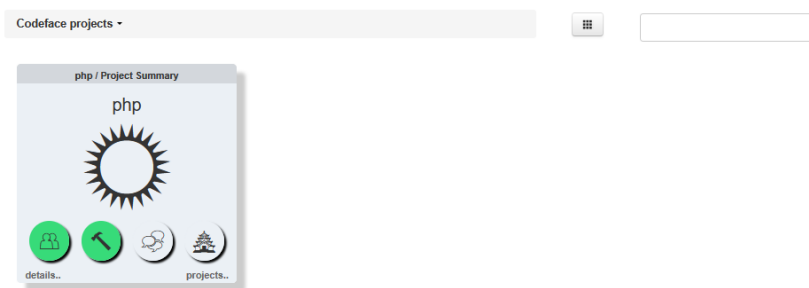


Fig. 1. Vista della dashboard.

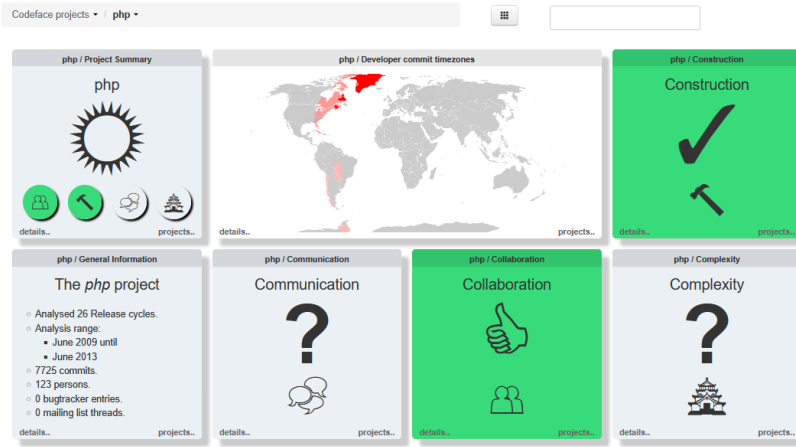


Fig. 2. Vista del progetto PHP.

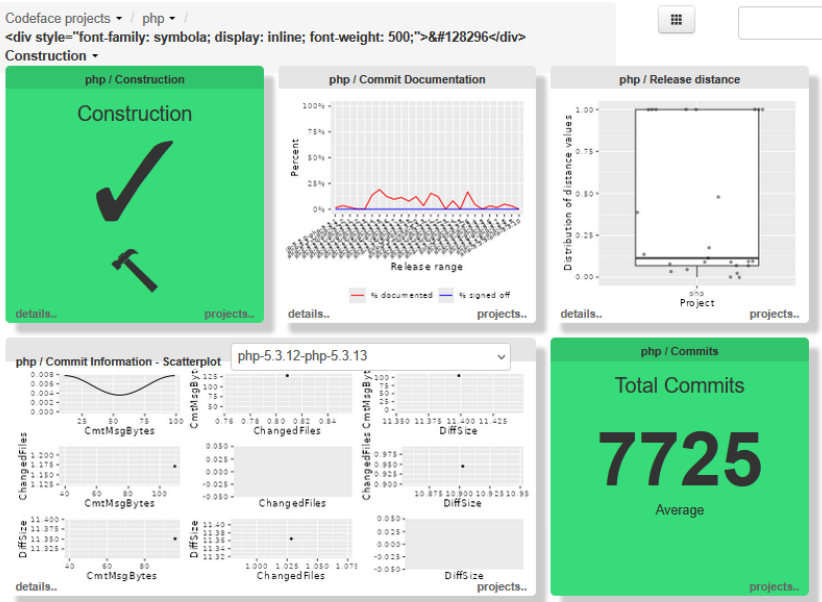


Fig. 3. Vista construction.

6.2 Integrazione continua e possibili estensioni

Un naturale sviluppo del lavoro svolto riguarda l'introduzione di pratiche di *Continuous Integration* (CI). Nel repository è già presente una pipeline di GitHub Actions dedicata all'ambiente Docker, che consente di verificare automaticamente la corretta build delle immagini e l'avviabilità dei servizi containerizzati.

A partire da questa base, ulteriori estensioni possibili includono:

- l'esecuzione automatica della test suite Python all'interno dei container;
- l'introduzione di job dedicati alla validazione degli script R;
- l'estensione della CI anche al workflow basato su Vagrant, ad esempio tramite build periodiche o verifiche di provisioning su runner dedicati.

L'adozione di una pipeline di integrazione continua più completa permetterebbe di intercettare regressioni in modo sistematico e di rendere il progetto più facilmente estendibile e manutenibile nel tempo, soprattutto in caso di evoluzioni future dello stack tecnologico o di contributi esterni.